Backpropagation

COMP6252 (Deep Learning Technologies)

ECS, University of Southampton

Computing the gradient

- An essential part of Deep Learning is computing the gradient of some loss function
- In most situations one cannot compute the gradient analytically
- PyTorch computes the gradient using three concepts
  1. Computational Graph
  2. Maintain a list of the derivatives of **primitive operations**
  3. Use the chain rule from calculus

Chain rule

- The chain rule allows us to compute the derivative of a composite function. Consider the following example

$$h = g(f(x))$$

- To compute $\frac{dh}{dx}$, the chain rule gives

$$\frac{dh}{dx} = \frac{dh}{dg}\frac{dg}{df}\frac{df}{dx}$$

- The crucial point in the above is that if we know each individual derivative then the result could be computed as the product.

Example

Let $f(x) = x^2$, $g(f) = \ln(f)$,$h(g) = g + 1$ with $x = 3$. We have:

$$\frac{df}{dx} = 2x = 6$$
$$\frac{dg}{df} = \frac{1}{f} = \frac{1}{9}$$
$$\frac{dh}{dg} = 1$$

Therefore

$$\frac{dh}{dx} = 1 \times \frac{1}{9} \times 6 = \frac{2}{3}$$

An important part of the above is that we know the derivative of **primitive operations/functions**: even the most complicated of expressions can be broken down into a sequence of primitive operations.

A second example

- In the previous example each function depended on a single variable
- What happens when a function depends on multiple variables?
  - Use partial derivatives
  - Sum the contributions from different variables
- Consider the following computational

$$a = 2$$

$$b = 4$$

$$c = a + b$$

$$d = \log(a) * \log(b) \text{ //not a primitive op. More later}$$

$$e = c * d$$

A second example

$$c = a + b \Rightarrow \frac{\partial c}{\partial a} = 1, \frac{\partial c}{\partial b} = 1$$

$$d = \log(a) * \log(b) \Rightarrow \frac{\partial d}{\partial a} = \frac{\log(b)}{a \ln(2)}, \frac{\partial d}{\partial b} = \frac{\log(a)}{b \ln(2)}$$

$$e = c * d \Rightarrow \frac{\partial e}{\partial c} = d, \frac{\partial e}{\partial d} = c$$

- Therefore

$$\frac{\partial e}{\partial a} = \frac{\partial e}{\partial c}\frac{\partial c}{\partial a} + \frac{\partial e}{\partial d}\frac{\partial d}{\partial a} = d \times 1 + c \times \frac{\log(b)}{a \ln(2)}$$

$$\frac{\partial e}{\partial b} = \frac{\partial e}{\partial c}\frac{\partial c}{\partial b} + \frac{\partial e}{\partial d}\frac{\partial d}{\partial b} = d \times 1 + c \times \frac{\log(a)}{b \ln(2)}$$

How can the chain rule help with automated differentiation?

- Now that we know about the chain rule, how can one use it to compute the gradient?
- Construct a graph where each primitive operation is represented by a node
- For each such node, attach auxillary nodes to compute its derivative
- As an example, let us see how one can automatically computes the following

$$a = 2$$

$$b = 4$$

$$c = a + b$$

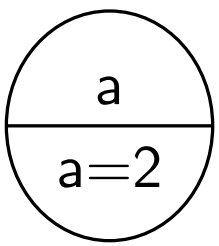
$$d = \log(a) * \log(b) \text{ //not a primitive op. More later}$$

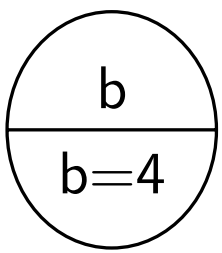
$$e = c * d$$

Computational Graph

$$a = 2$$

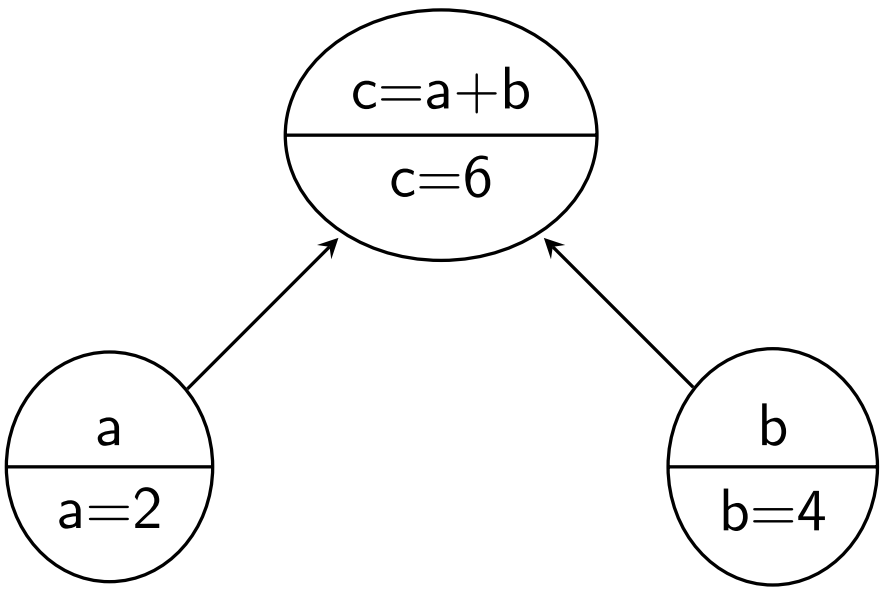
$$b = 4$$



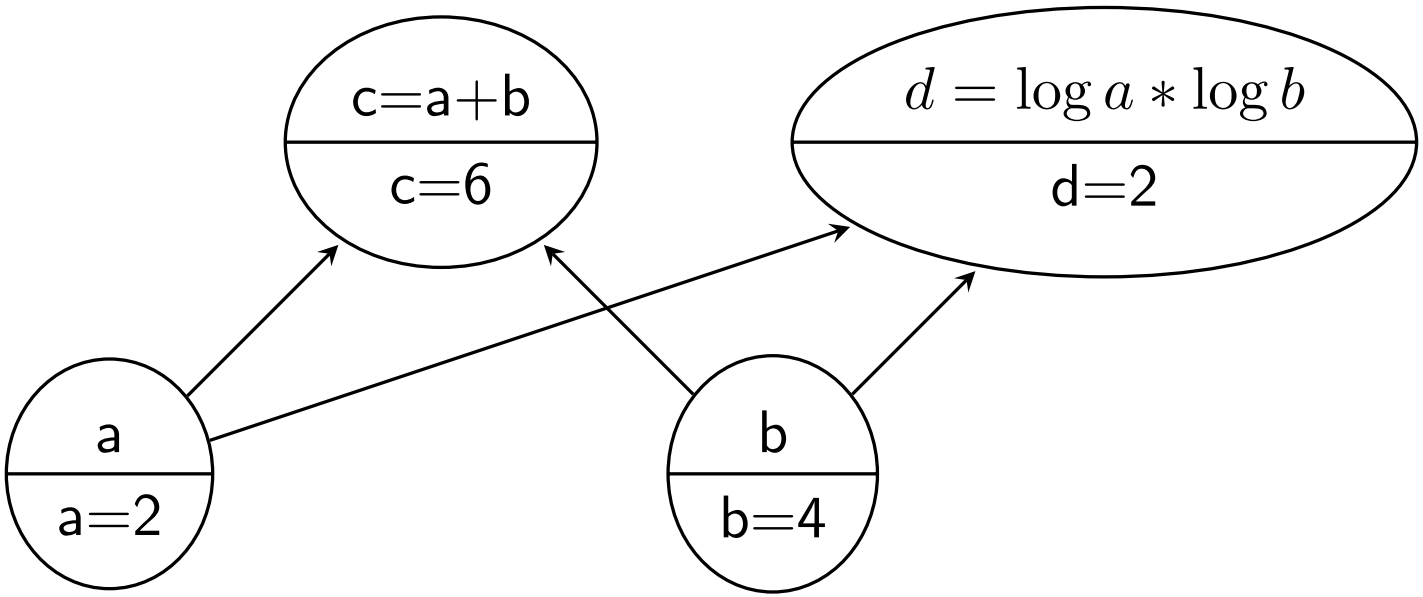


Computational Graph

$$a = 2$$
$$b = 4$$
$$c = a + b$$



Computational Graph



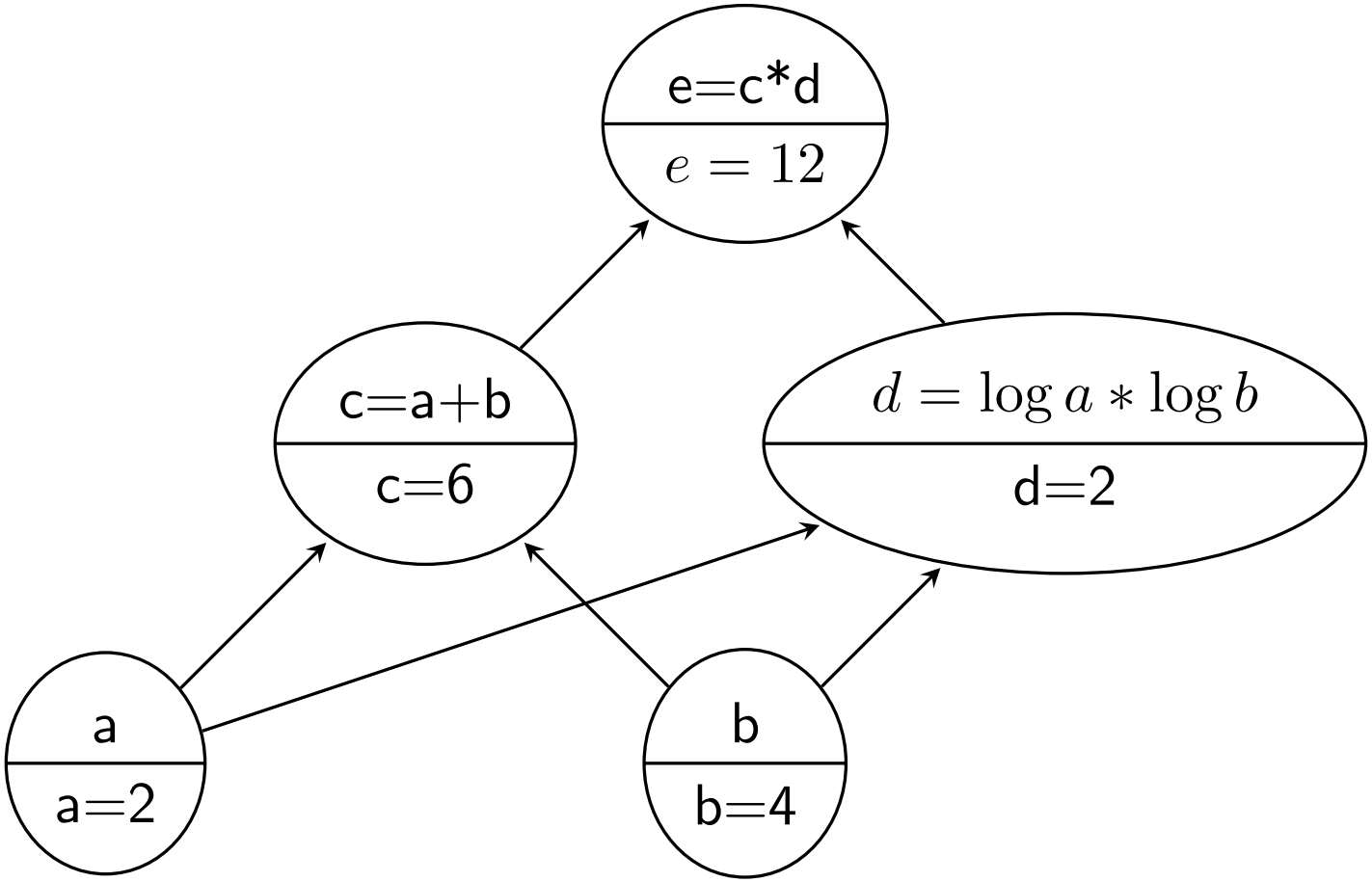
$$a = 2$$

$$b = 4$$

$$c = a + b$$

$$d = \log(a) * \log(b)$$

Computational Graph

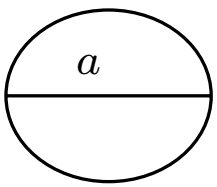


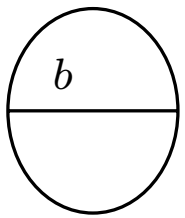
$$a = 2$$
$$b = 4$$
$$c = a + b$$
$$d = \log(a) * \log(b)$$
$$e = c * d$$

How does the construction of the computational graph help in computing the gradients?

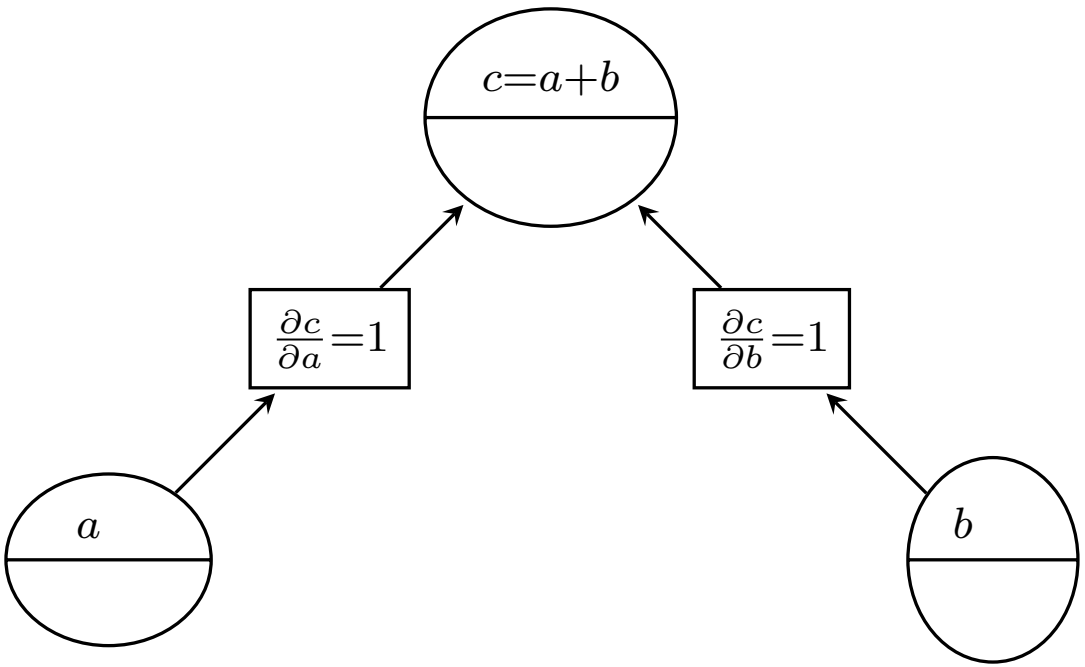
- While constructing the computational graph, auxillary nodes representing the derivatives are added
- Since the graph is a sequence of primitive operations the result of such "local derivatives" can be computed as the graph is constructed
- For example if $z = x + y$, we know that $\frac{\partial z}{\partial x}$=1 and $\frac{\partial z}{\partial y}$=1
- Therefore when node z is created, nodes for $\frac{\partial z}{\partial x}$=1 and $\frac{\partial z}{\partial y}$=1 are also created
- We refer to this phase as "forward pass"

forward pass

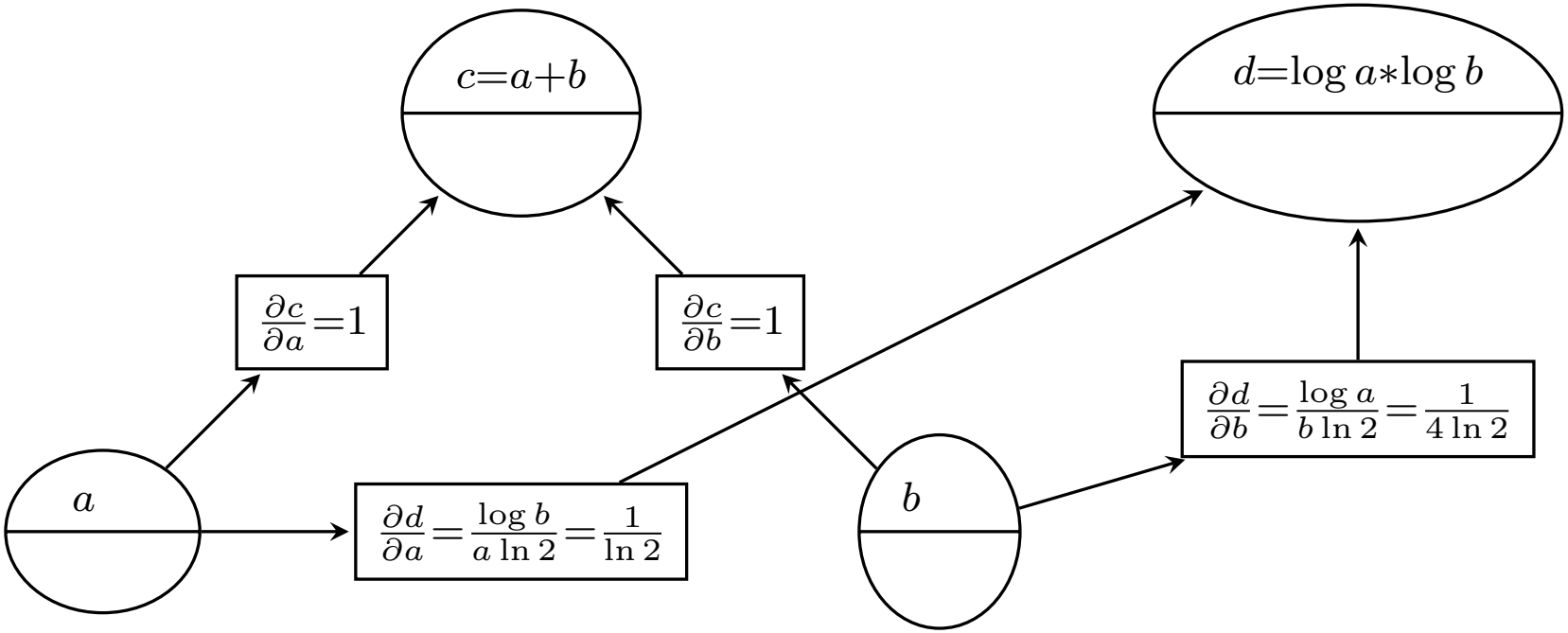




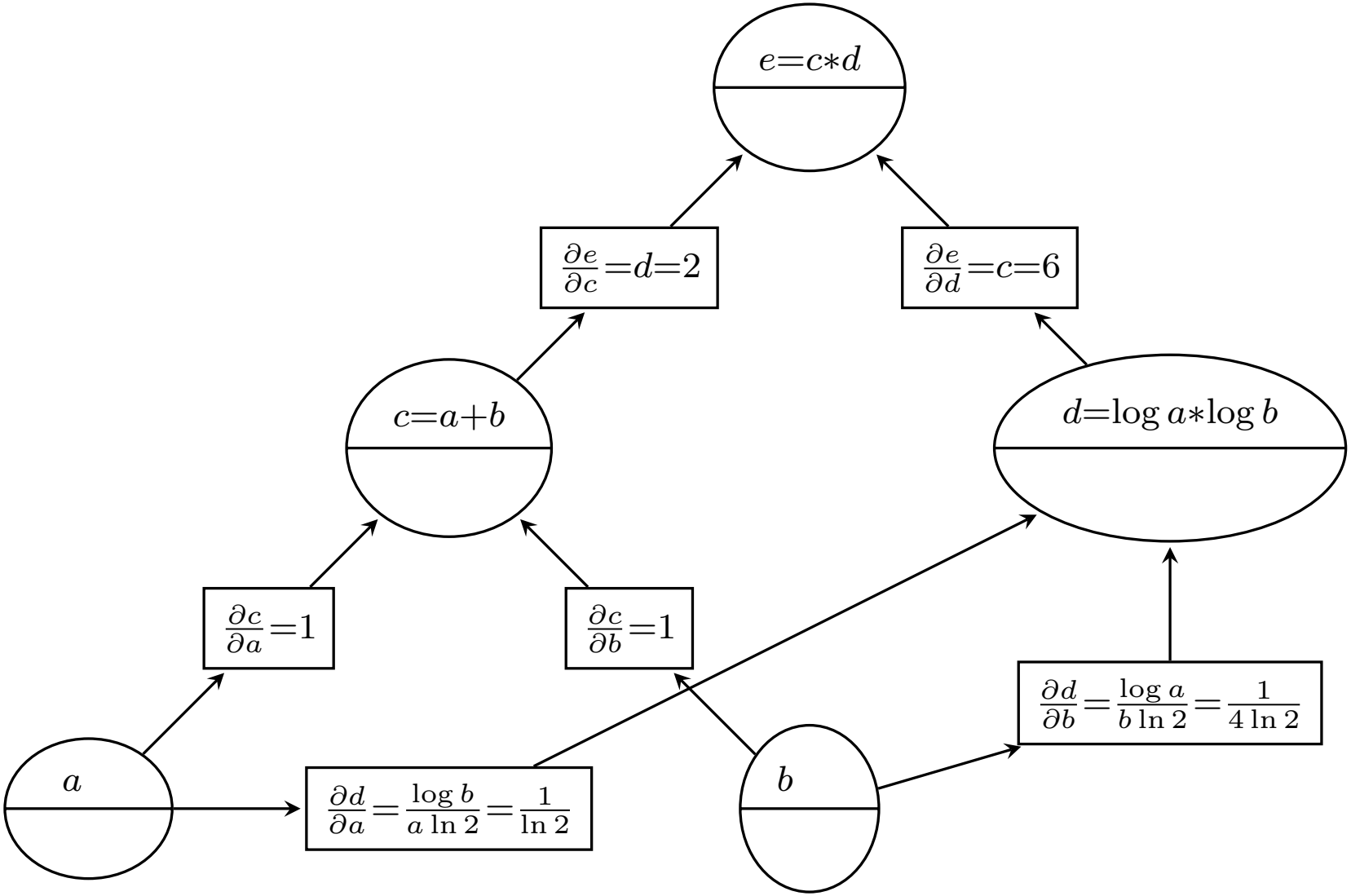
forward pass



forward pass



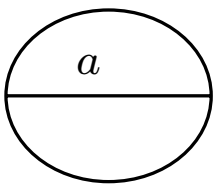
forward pass

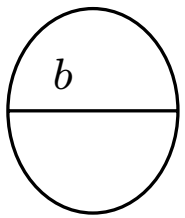


Eager vs Graph mode

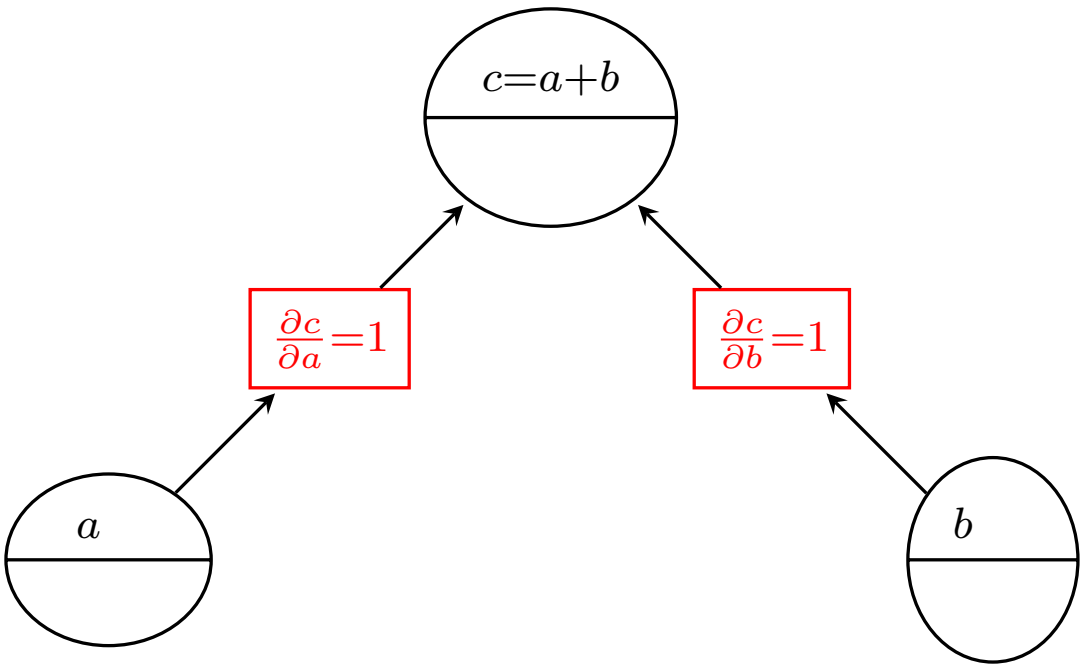
- Most ML frameworks support two modes of execution
  1. Eager mode: operators are executed immediately. This is PyTorch default
  2. Graph mode: the computation starts **after** the whole graph for the forward pass is built
- In eager mode **no forward pass graph is built**
- In our example we built forward pass graphs for purely illustrative purposes
- The graph for computing the gradients (backward pass) **is built**
- Next we describe how the backward pass uses the graph to compute the gradients
- The nodes that are actually created in eager mode are **colored in red**

Eager mode forward pass

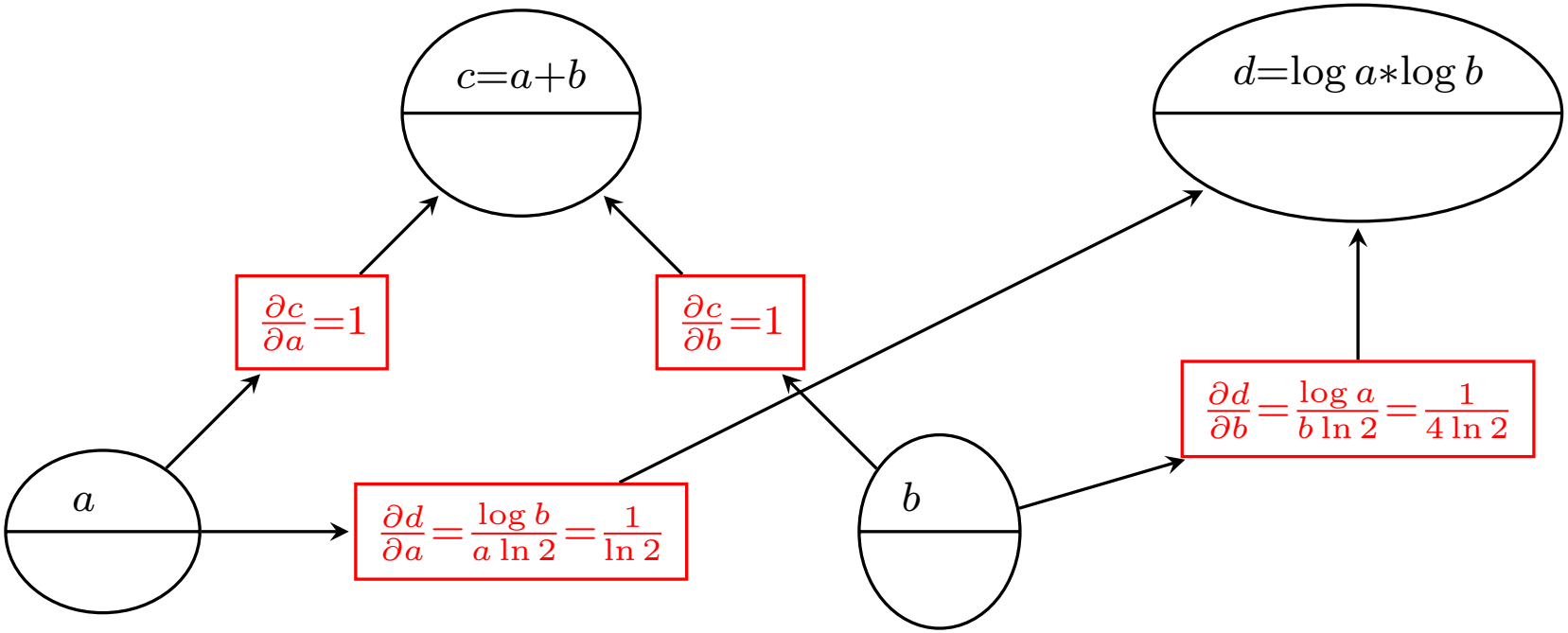




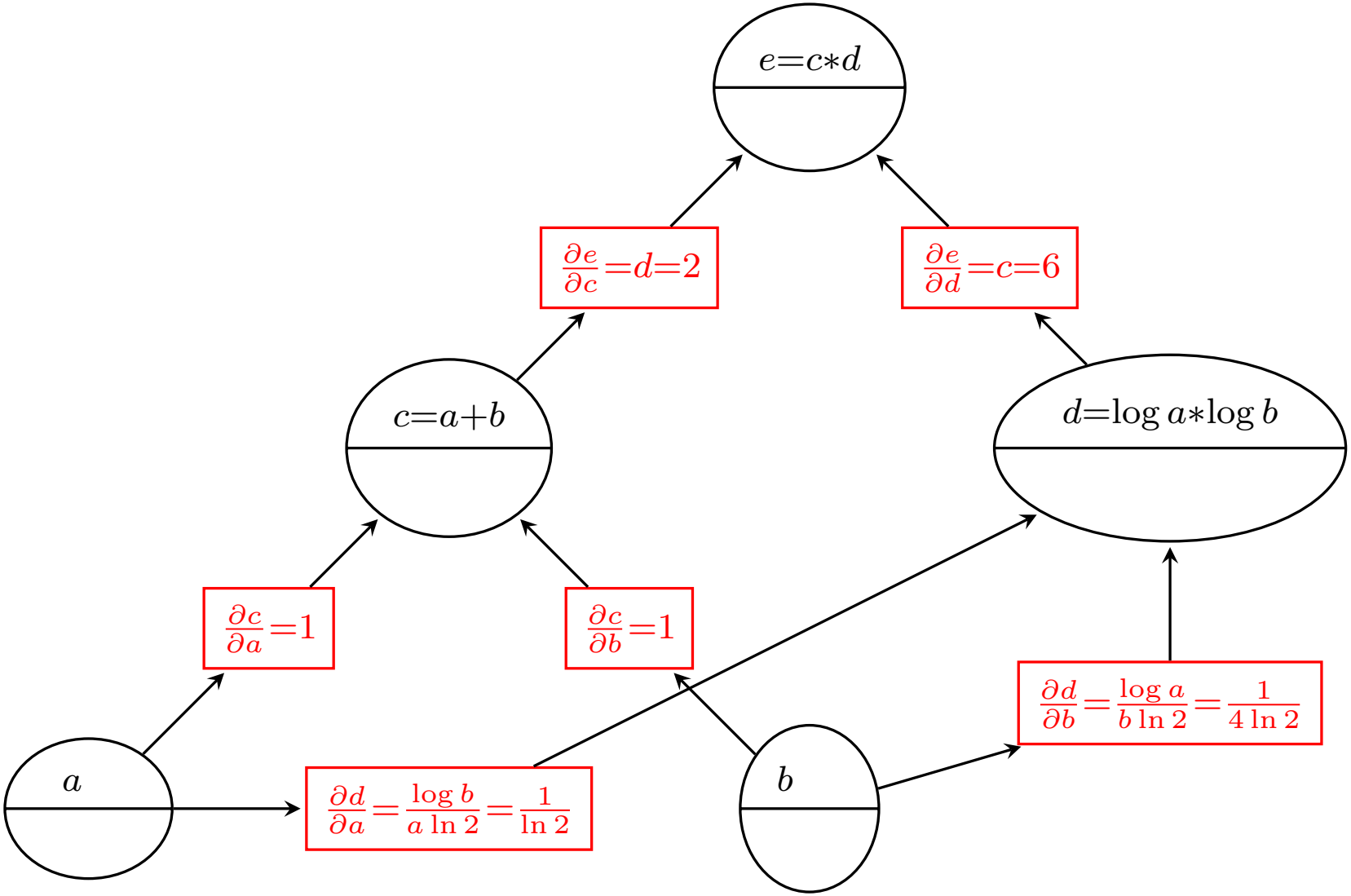
Eager mode forward pass



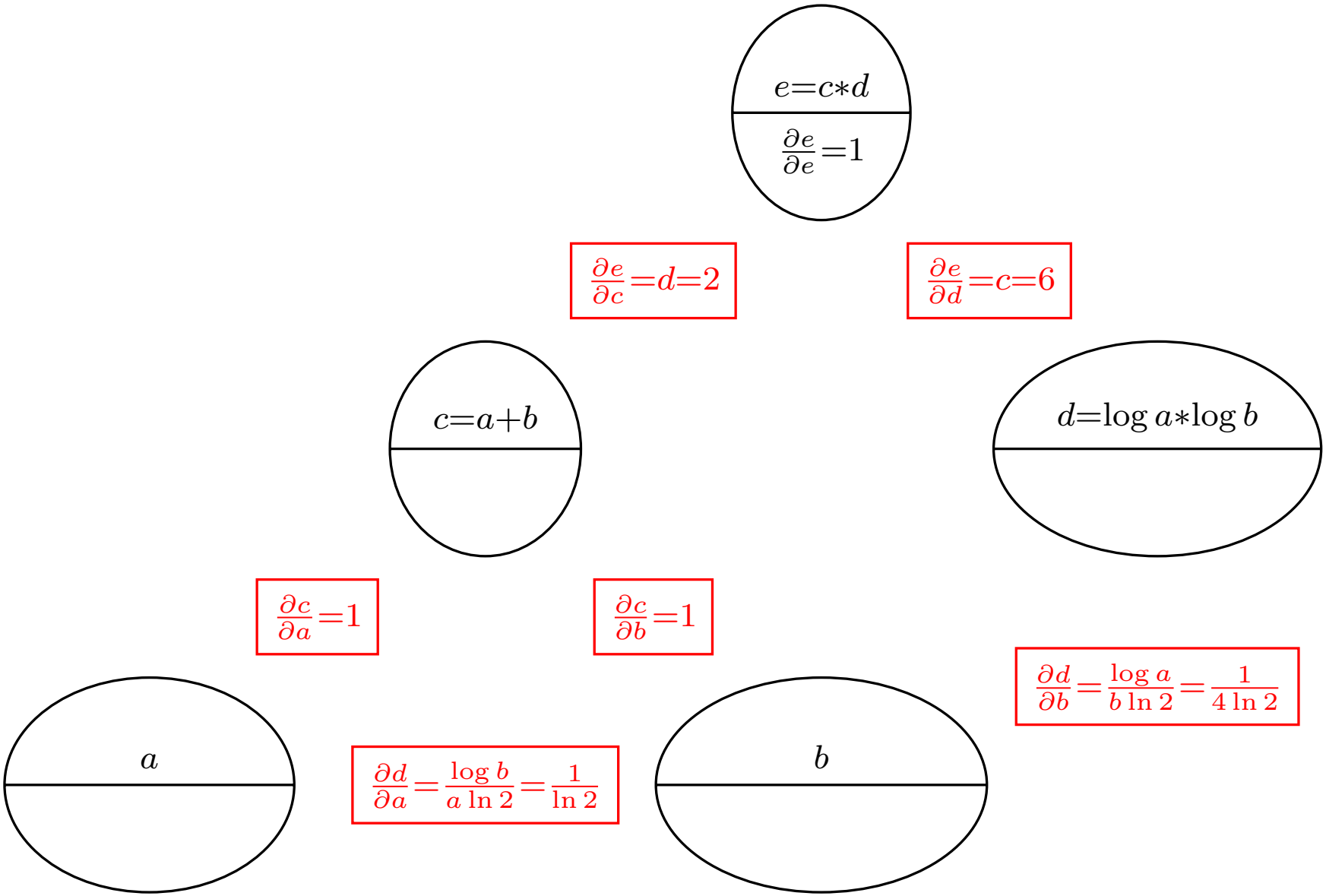
Eager mode forward pass



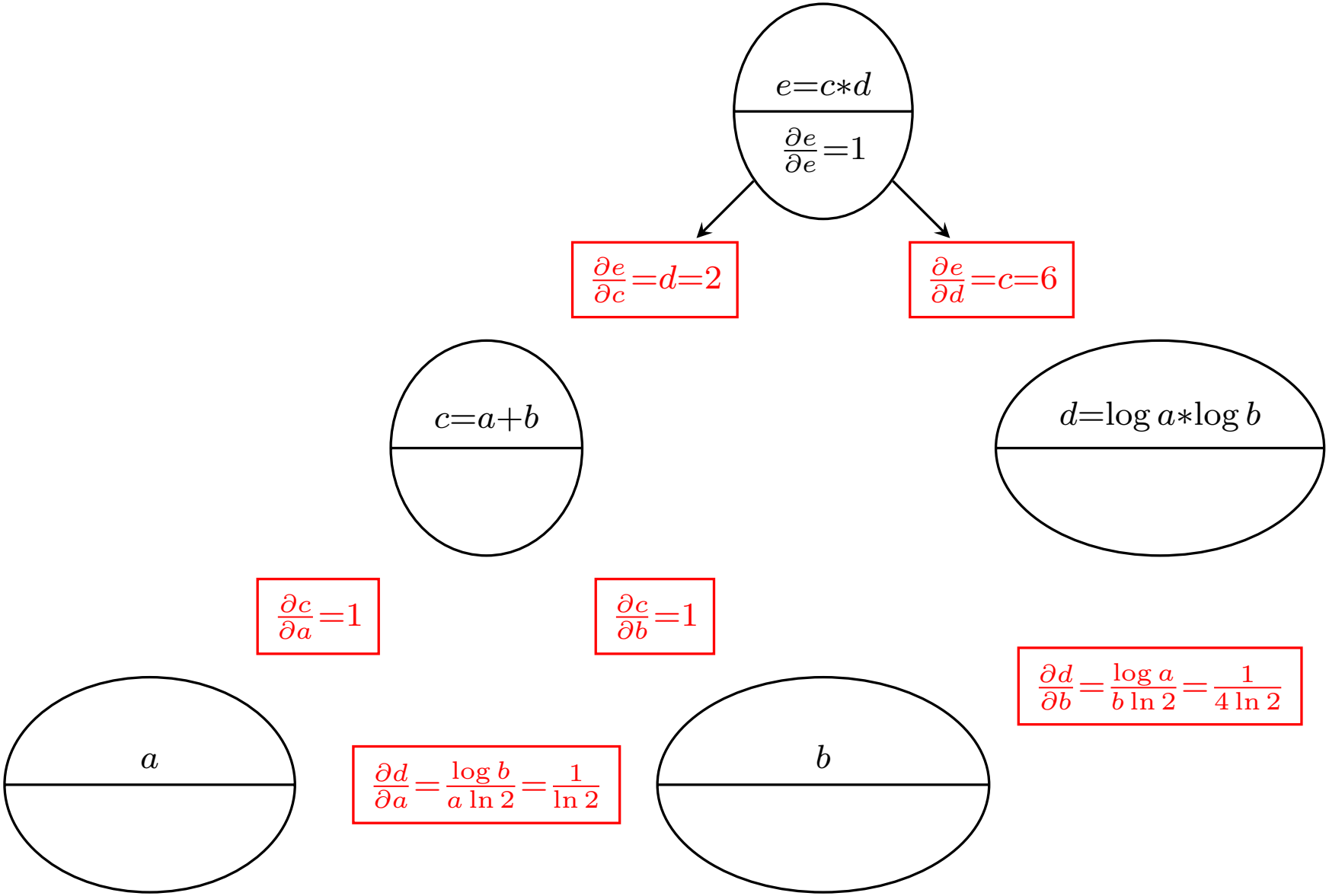
Eager mode forward pass



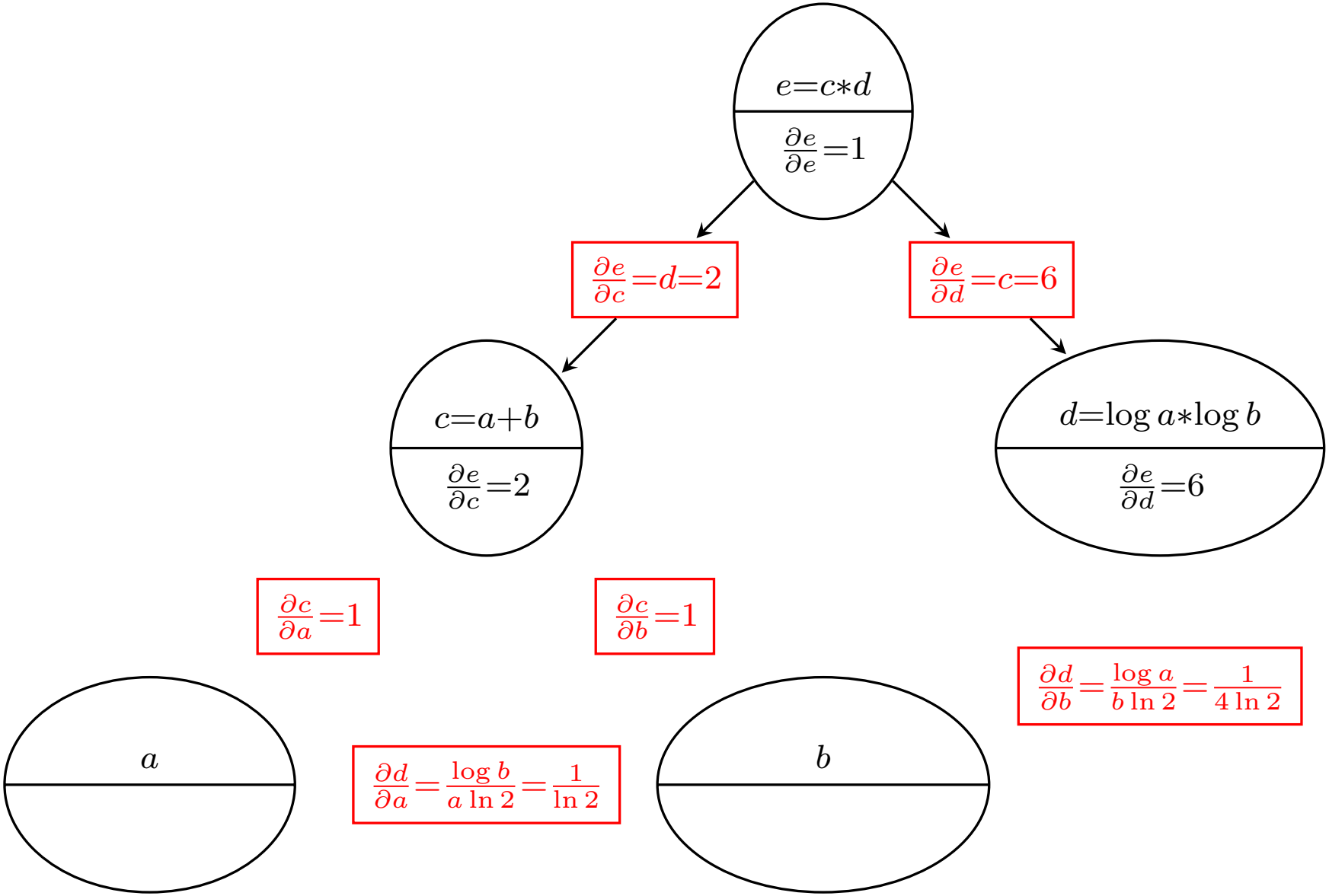
Backward pass



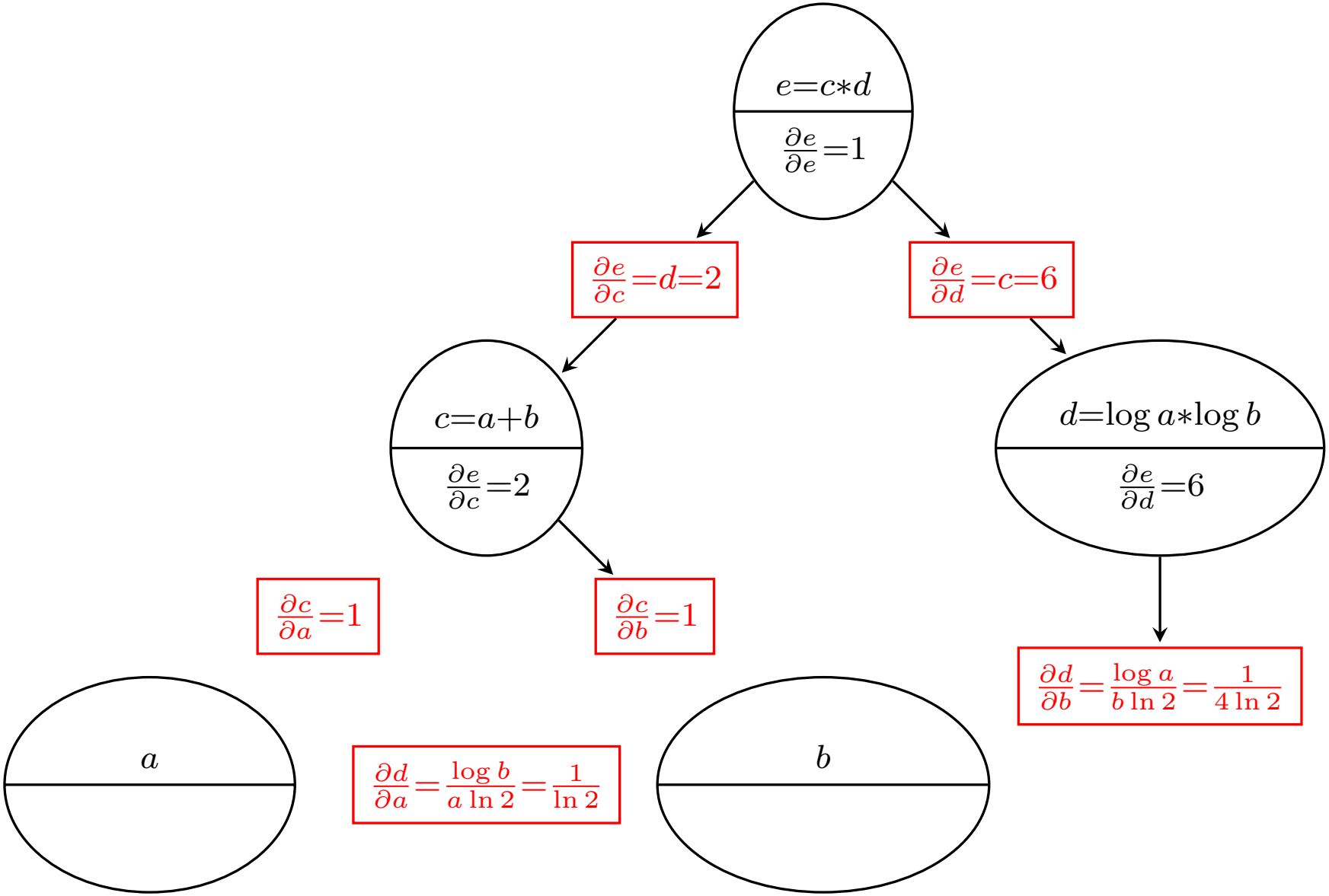
Backward pass



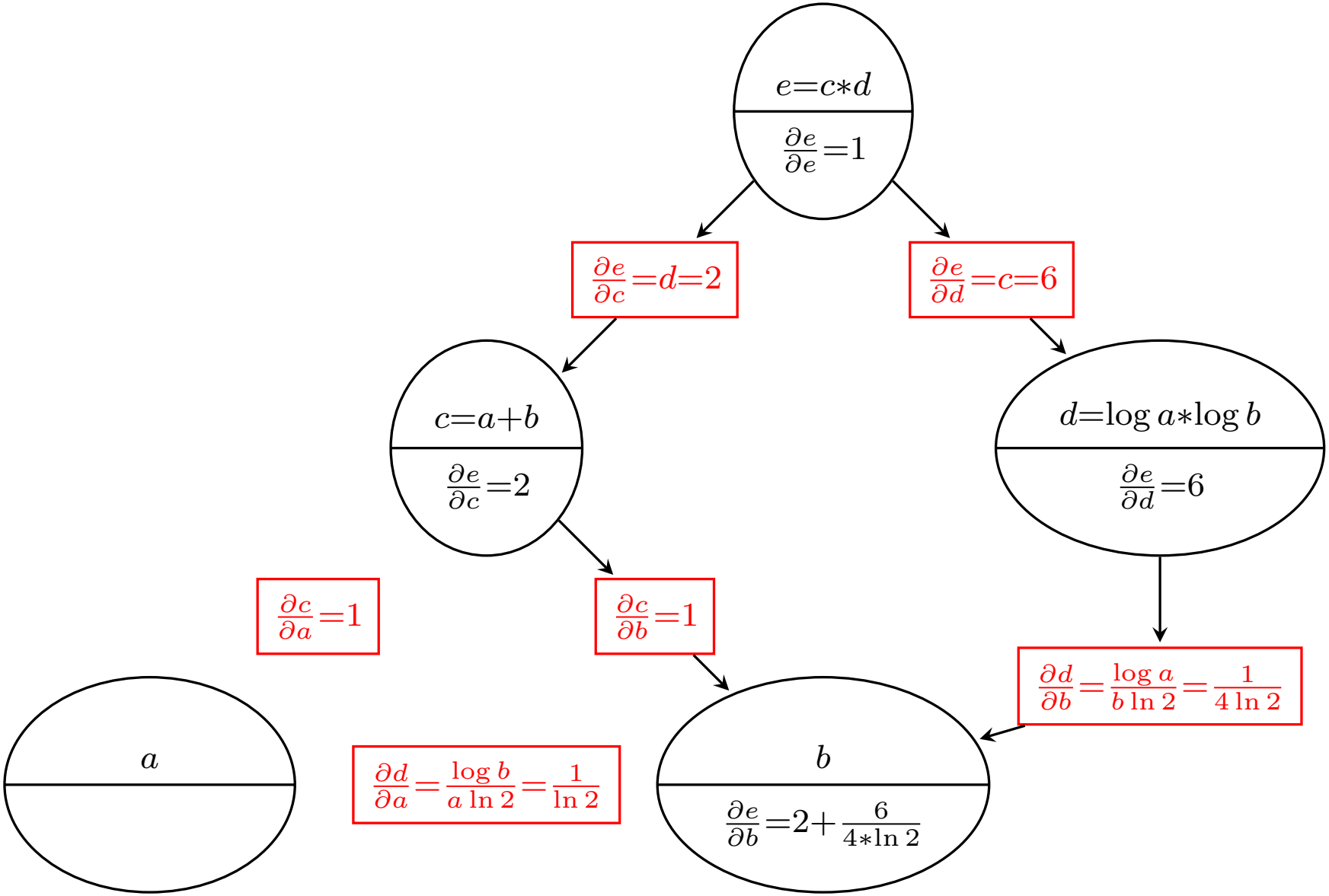
Backward pass



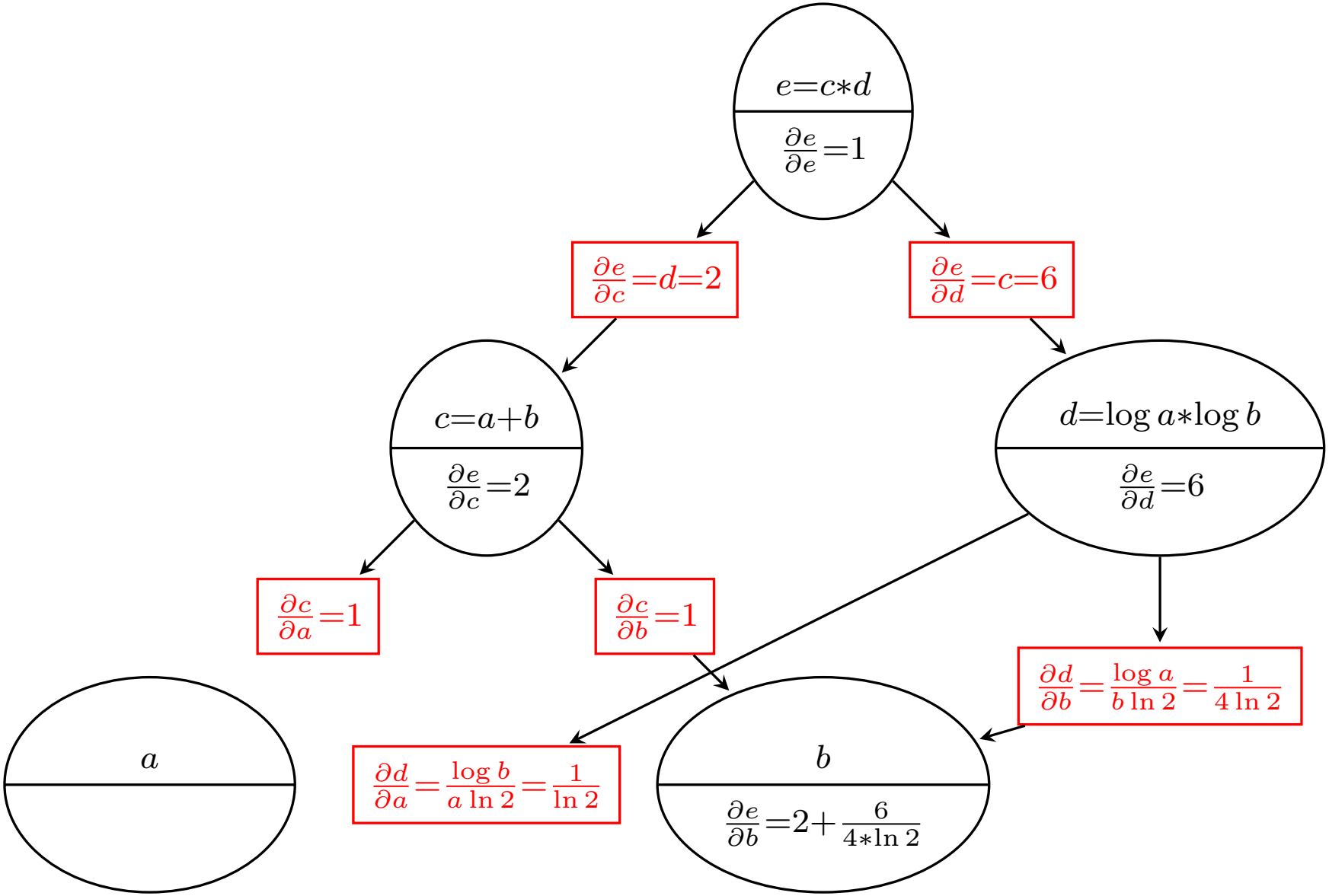
Backward pass



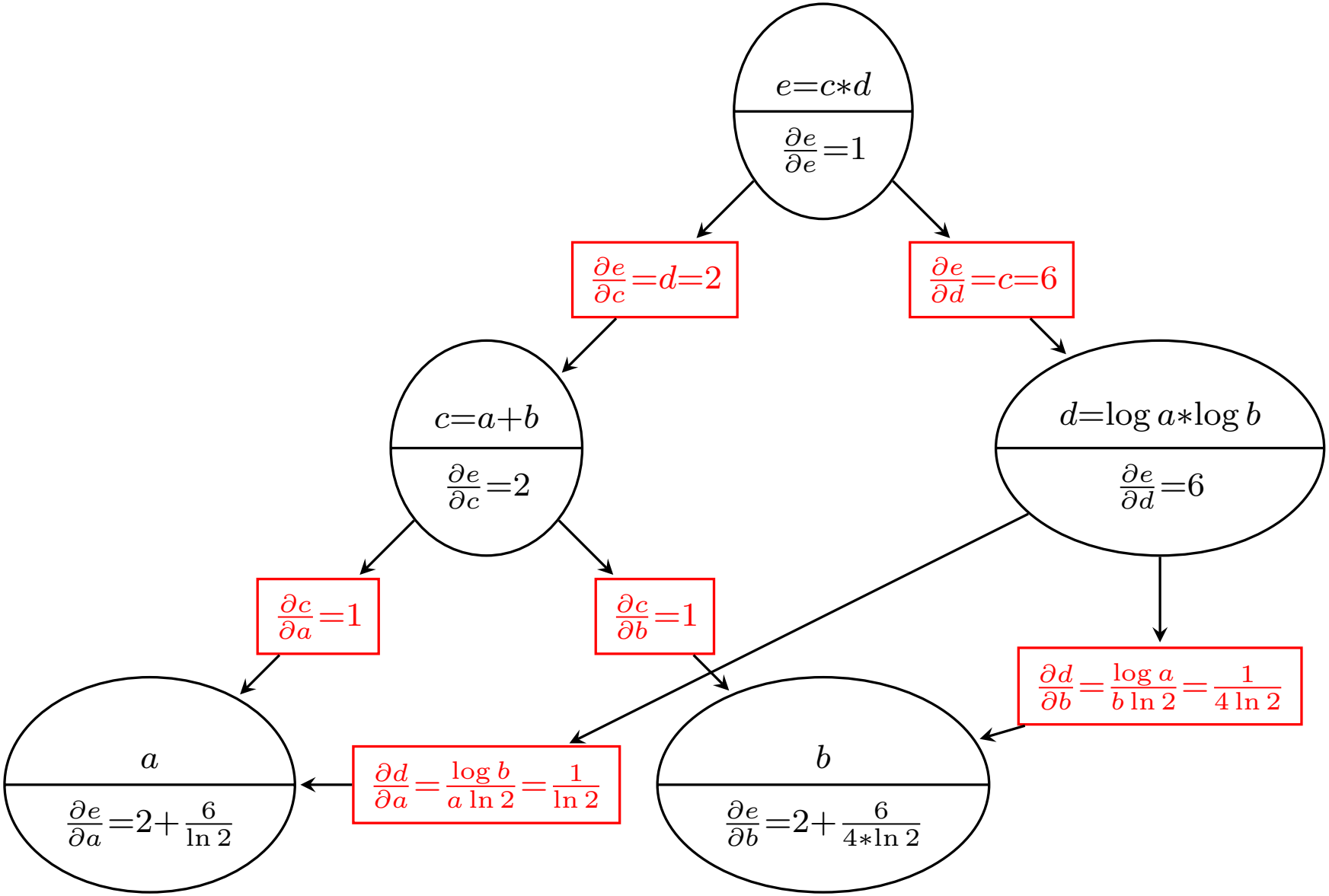
Backward pass



Backward pass



Backward pass



Details for node d

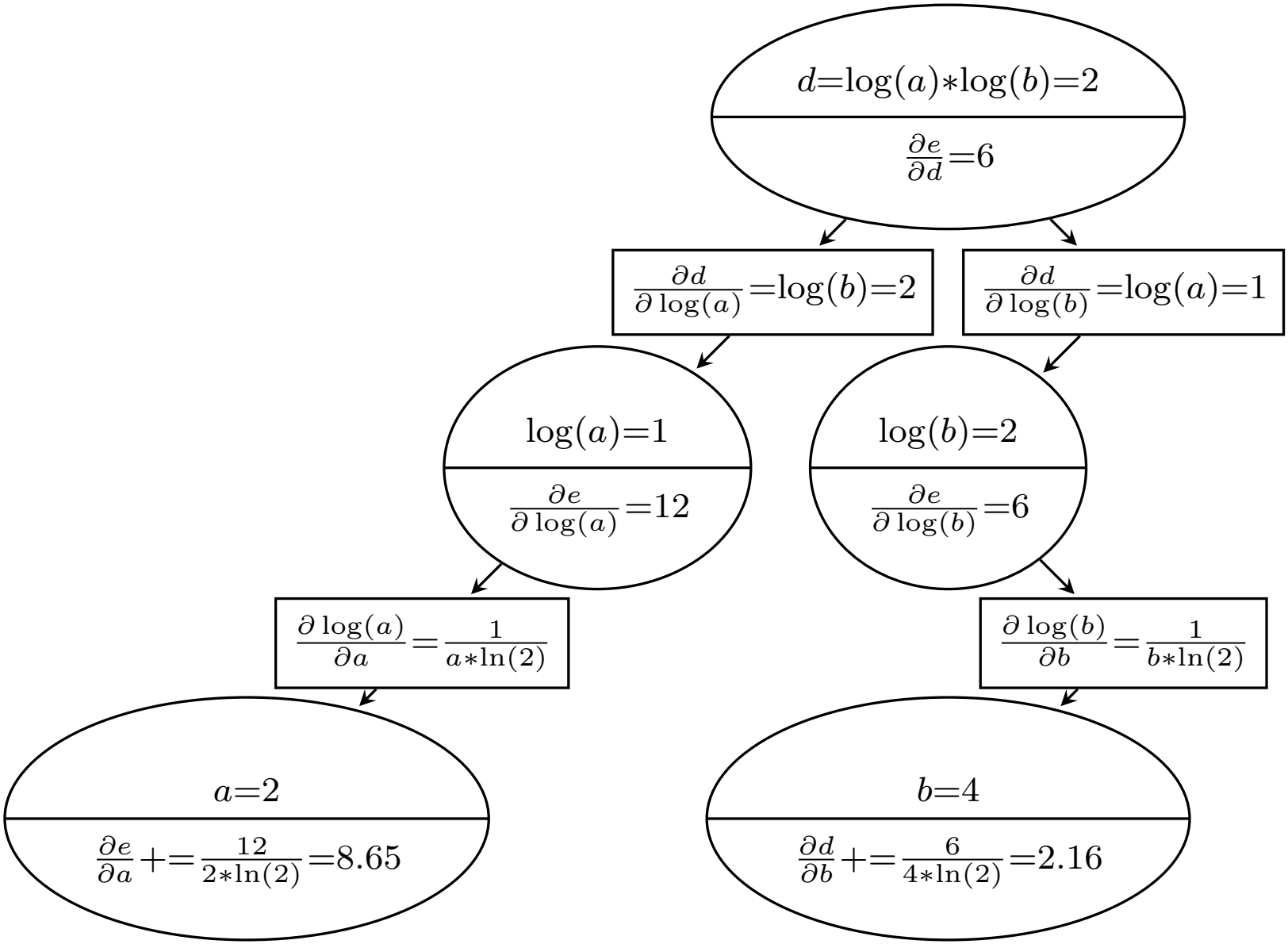
- For simplicity and slide space consideration the computation for node d was somehow condensed
- $d = \log(a) * \log(b)$ **is not a primitive operation**
- An ML framework would actually break it down into a sequence of primitive operations by creating temporary, unnamed, nodes

$$t_1 = \log(a) \Rightarrow \frac{\partial t_1}{\partial a} = \frac{1}{a \ln(2)}$$

$$t_2 = \log(b) \Rightarrow \frac{\partial t_2}{\partial b} = \frac{1}{b \ln(2)}$$

$$d = t_1 * t_2 \Rightarrow \frac{\partial d}{\partial t_1} = t_2, \frac{\partial d}{\partial t_2} = t_1$$

Details for node d

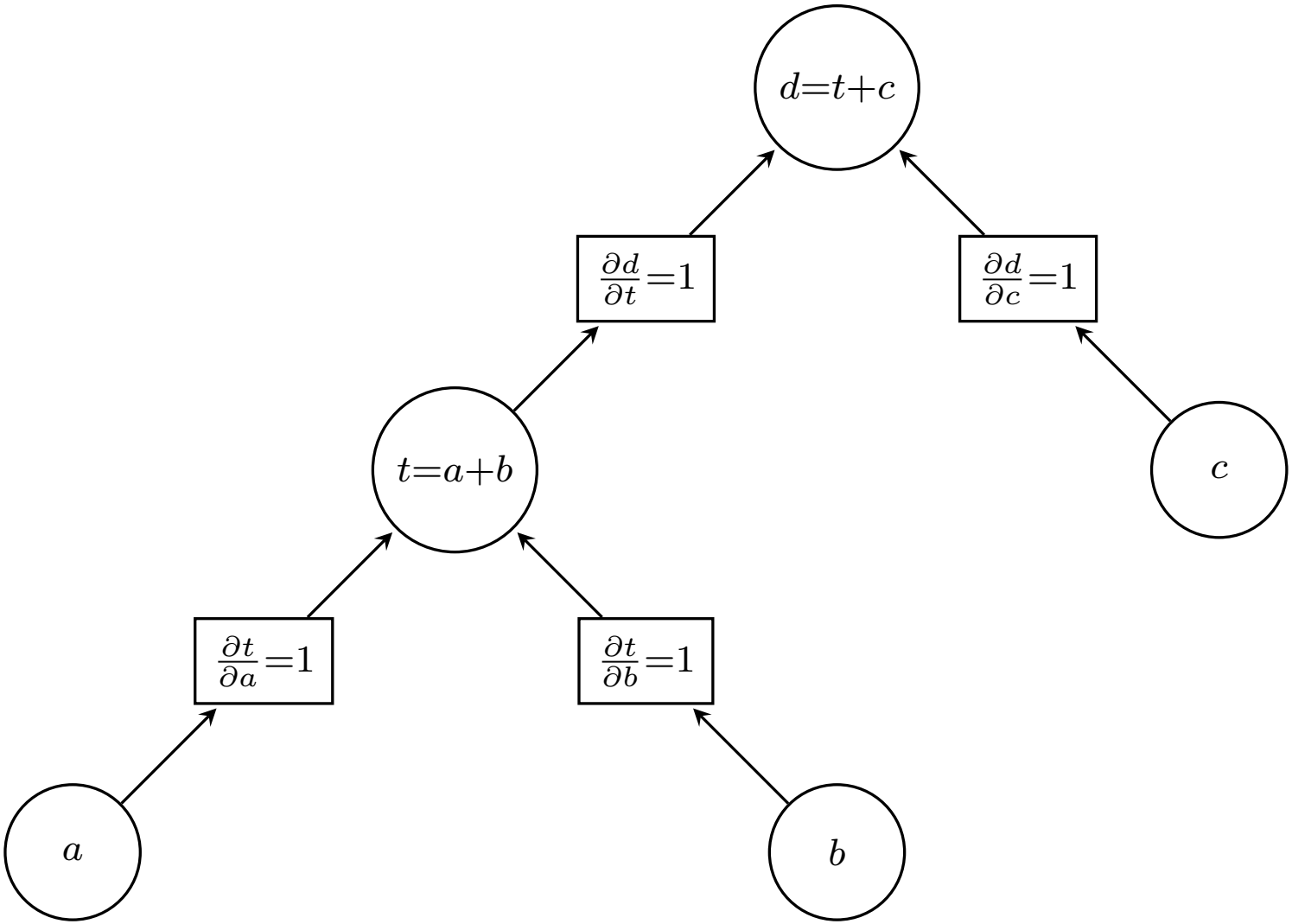


Example

to compute $d = a + b + c$? in graph mode,

1. how many nodes are needed (without gradient nodes)
2. Including the gradient nodes?

Graph for $d = a + b + c$



PyTorch Details

- PyTorch implements the above using a graph of gradient functions: `grad_fn`
- The values in rectangular nodes represent the outputs of those functions
- The starting point is `e.grad_fn` with input 1. The output of `e.grad_fn(1.)` is `[2,6]` as expected.
- The output `[2,6]` is used as input to the functions `e.grad_fn.next_functions`
- The code for the whole graph is shown on the corresponding notebook